Instituto Politécnico Nacional
Centro de Investigación en Computación

Clasificación Inteligente de Patrones (A25)
Prof.: Dr. Antonio Alarcón Paredes

Tarea 7

Luis Angel Rojo Chavez
arojocz@gmail.com
lrojoc2024@cic.ipn.mx

06 de abril de 2025

Árboles de decisión

Programar un árbol de decisión simple, como el visto en clase, que sea aplicado a las clases 1 y 2 (NO linealmente separables) del dataset de iris.Después, aplicarlo a los otros dos datasets que descargaron usando hold-out estratificado con partición 70/30.

Siéntanse libres de modificar su árbol incorporando alguna técnica que permita limitar en profundidad el mismo, o bien modificando el criterio de impureza de Gini por alguno otro.

Descripción

Se utiliza la medida de impureza de gini, se definió una función gini que recibe de entrada los patrones y las clases y orden cada columna, obtiene los promedios consecutivos y calcula el gini mínimo de cada fila para luego guardar el valor mínimo para decidir qué columna es la mejor o del gini menor (menos impura) y el umbral para dividir los datos.

$$\text{Gini}(D) = 1 - \sum_{i=1}^{k} p_i^2 \tag{1}$$

El arbol de decisión usa la función Gini para dividir en dos ramas los datos (rama true izquierda o false derecha) y hace llamadas recursivas para formar los subárboles o nodos del árbol hasta llegar al caso base que es que el nodo no tenga impurezas o también llegar a la profundidad máxima definida que en el programa es hasta 6.

Código

```
from sklearn.model_selection import train_test_split
import pandas as pd
import numpy as np
from sklearn.metrics import balanced_accuracy_score, recall_score
from sklearn.metrics import confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns

datasets = ['iris.data', 'heart.dat', 'led7digit.dat']
```

```python
def read_dataset(dataset,  delimiter=',', header=None):
        df = pd.read_csv(dataset, comment='@')

        X = df.iloc[:, :-1].values
        y = df.iloc[:, -1].values

        # binary classes only
        if dataset == 'iris.data':
            mask = y != 'Iris-setosa'
            X = X[mask]
            y = y[mask]
            y = np.where(y == 'Iris-versicolor', 0, 1)
            #y -= 1  # 0 y 1
        else:
            mask = (y > 0) & (y < 3)
            X = X[mask]
            y = y[mask]
            y -= 1  # 0 y 1

        # Hold-out 70/30 stratified
        X_train, X_test, y_train, y_test = train_test_split(
            X, y, test_size=0.3 , stratify=y, random_state=42
        )
        return X, y, X_train, X_test, y_train, y_test

def gini(X, y):
    #sort each column and average of every 2 consecutive values
    min_values = np.array([])
    best_thresholds = []
    for i in range(X.shape[1]):
        gini_list = []
        column = X[:,i]
        index_sort = np.argsort(column) #sorted indexes
        column_sorted = column[index_sort]
        y_sorted = y[index_sort]
        averages = (column_sorted[1:] + column_sorted[:-1]) / 2
        for average in averages:
            left_leaf = column_sorted < average
            right_leaf = column_sorted >= average

            left_leaf = y_sorted[left_leaf]
            right_leaf = y_sorted[right_leaf]

            #calculate gini
            mask1 = left_leaf == 0
```

```
            mask2 = right_leaf == 0

            if len(left_leaf) > 0:
                p1 = len(left_leaf[mask1]) / len(left_leaf)
                p2 = len(left_leaf[~mask1]) / len(left_leaf)
                gini_1 = 1 - p1**2 - p2**2
            else:
                gini_1 = 1

            if len(right_leaf) > 0:
                p1 = len(right_leaf[mask2]) / len(right_leaf)
                p2 = len(right_leaf[~mask2]) / len(right_leaf)
                gini_2 = 1 - p1**2 - p2**2
            else:
                gini_2 = 1

            gini = gini_1*(len(left_leaf)/len(column_sorted)) + gini_2*(len(right_leaf)/len(column_sorte
            gini_list.append(gini)

        gini_array = np.array(gini_list)
        min_gini = np.argmin(gini_array)
        best_thresholds.append(averages[min_gini])
        min_values = np.append(min_values, gini_array[min_gini])

    best_column = np.argmin(min_values)
    best_threshold = best_thresholds[best_column]
    return best_column, best_threshold, min_values[best_column]

def tree(X, y, depth = 0):
    # base case
    if len(set(y)) == 1 or depth == 6:
        # leaf with majority class
        return {'type': 'leaf', 'class': y[0], 'gini': 0}  # gini = 0 porque todas las clases son igual

    # best división with Gini
    best_column, best_threshold, min_gini = gini(X, y)

    # branches
    left_mask = X[:, best_column] < best_threshold
    right_mask = ~left_mask

    # base case 2
    if left_mask.sum() == 0 or right_mask.sum() == 0:
        return {'type': 'leaf', 'class': np.bincount(y).argmax(), 'gini': min_gini}
```

```
    # divide the dataset
    X_left, y_left = X[left_mask], y[left_mask]
    X_right, y_right = X[right_mask], y[right_mask]

    # recursively create subtrees
    left_subtree = tree(X_left, y_left, depth = depth +1)
    right_subtree = tree(X_right, y_right, depth= depth + 1)

    # create the node with divisions, subtres and gini
    return {'type': 'internal',
            'column': best_column,
            'threshold': best_threshold,
            'left': left_subtree,
            'right': right_subtree,
            'gini': min_gini
    }

def print_tree(node, depth=0):
    indent = "  " * depth
    if node['type'] == 'leaf':
        print(f"{indent}Leaf: class = {node['class']} (gini = {node['gini']:.4f})")
    else:
        print(f"{indent}Node: X[{node['column']}] < {node['threshold']:.3f} (gini = {node['gini']:.4f})'
        print_tree(node['left'], depth + 1)
        print_tree(node['right'], depth + 1)


def decision_tree(tree_model, x):
    while tree_model['type'] != 'leaf':
        if x[tree_model['column']] < tree_model['threshold']:
            tree_model = tree_model['left']
        else:
            tree_model = tree_model['right']
    return tree_model['class']

for i in datasets:
    print(f'Clasificación para {i}')
    X, y, X_train, X_test, y_train, y_test = read_dataset(i)
    #gini(X_train, y_train)
    print('-'*100)
    tree_built = tree(X_train, y_train)
    #pred = decision_tree(tree_built, X_test[0])
    y_pred = [decision_tree(tree_built, x) for x in X_test]
    counter = 0
    for element in y_pred:
```

```
        print(f"Clasificador: {element}, Verdadero: {y_test[counter]}")
        counter += 1

    balanced = balanced_accuracy_score(y_test, y_pred)
    sensitivity = recall_score(y_test, y_pred, pos_label=1)
    specificity = recall_score(y_test, y_pred, pos_label=0)
    print('')
    print(f"Balanced Accuracy: {balanced:.4f}")
    print(f"Sensibilidad (clase 1): {sensitivity:.4f}")
    print(f"Especificidad (clase 0): {specificity:.4f}")
    print('')
    print_tree(tree_built)
    print('='*90)
    cm = confusion_matrix(y_test, y_pred)
    plt.figure(figsize=(5,4))
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
                xticklabels=['Clase 0', 'Clase 1'],
                yticklabels=['Clase 0', 'Clase 1'])
    plt.xlabel('Predicción')
    plt.ylabel('Valor verdadero')
    plt.title(f'Matriz de Confusión de {i}')
    plt.show()
```

Para las evidencias de los resultados se presentan los resultados del hold-out 70/30 y también con el dataset completo, se presenta como resultado en el programa el Balanced accuracy, sensibilidad y especificidad. En los anexos se presentan las matrices de confusión de cada dataset para ambos casos (hold-out y dataset completo).

Iris

```
Balanced Accuracy: 0.8667
Sensibilidad (clase 1): 0.8000
Especificidad (clase 0): 0.9333

Node: X[3] < 1.750 (gini = 0.0541)
  Node: X[2] < 5.050 (gini = 0.0360)
    Leaf: class = 0 (gini = 0.0000)
    Node: X[0] < 6.150 (gini = 0.0000)
      Leaf: class = 0 (gini = 0.0000)
      Leaf: class = 1 (gini = 0.0000)
  Leaf: class = 1 (gini = 0.0000)
```

Figura 1: Representación del árbol y sus métricas de evaluación.

```
Balanced Accuracy: 1.0000
Sensibilidad (clase 1): 1.0000
Especificidad (clase 0): 1.0000

Node: X[3] < 1.750 (gini = 0.1103)
  Node: X[2] < 4.950 (gini = 0.0856)
    Node: X[3] < 1.650 (gini = 0.0000)
      Leaf: class = 0 (gini = 0.0000)
      Leaf: class = 1 (gini = 0.0000)
    Node: X[3] < 1.550 (gini = 0.2222)
      Leaf: class = 1 (gini = 0.0000)
      Node: X[0] < 6.950 (gini = 0.0000)
        Leaf: class = 0 (gini = 0.0000)
        Leaf: class = 1 (gini = 0.0000)
  Node: X[2] < 4.850 (gini = 0.0290)
    Node: X[0] < 5.950 (gini = 0.0000)
      Leaf: class = 0 (gini = 0.0000)
      Leaf: class = 1 (gini = 0.0000)
    Leaf: class = 1 (gini = 0.0000)
```

Figura 2: Representación del árbol y sus métricas de evaluación para el dataset completo.

Heart Disease

```
Node: X[12] < 4.500 (gini = 0.3576)
  Node: X[2] < 3.500 (gini = 0.2704)
    Node: X[4] < 154.500 (gini = 0.1308)
      Leaf: class = 1 (gini = 0.0000)
      Node: X[9] < 31.000 (gini = 0.1077)
        Node: X[0] < 57.000 (gini = 0.0966)
          Leaf: class = 0 (gini = 0.0000)
          Node: X[1] < 0.500 (gini = 0.2000)
            Leaf: class = 0 (gini = 0.0000)
            Leaf: class = 0 (gini = 0.0000)
        Leaf: class = 1 (gini = 0.0000)
    Node: X[11] < 0.500 (gini = 0.3450)
      Node: X[3] < 145.000 (gini = 0.1655)
        Node: X[4] < 316.500 (gini = 0.0625)
          Leaf: class = 0 (gini = 0.0000)
          Node: X[0] < 59.000 (gini = 0.0000)
            Leaf: class = 0 (gini = 0.0000)
            Leaf: class = 1 (gini = 0.0000)
        Node: X[0] < 49.000 (gini = 0.0000)
          Leaf: class = 0 (gini = 0.0000)
          Leaf: class = 1 (gini = 0.0000)
      Node: X[1] < 0.500 (gini = 0.1000)
        Node: X[3] < 134.000 (gini = 0.0000)
          Leaf: class = 0 (gini = 0.0000)
          Leaf: class = 1 (gini = 0.0000)
        Leaf: class = 1 (gini = 0.0000)
  Node: X[11] < 0.500 (gini = 0.2687)
    Node: X[2] < 3.500 (gini = 0.3688)
      Node: X[4] < 272.000 (gini = 0.1875)
        Leaf: class = 0 (gini = 0.0000)
        Node: X[1] < 0.500 (gini = 0.2500)
          Leaf: class = 0 (gini = 0.0000)
          Node: X[0] < 56.500 (gini = 0.0000)
            Leaf: class = 0 (gini = 0.0000)
            Leaf: class = 1 (gini = 0.0000)
      Node: X[0] < 63.000 (gini = 0.3111)
        Node: X[7] < 159.500 (gini = 0.2359)
          Node: X[7] < 112.500 (gini = 0.1026)
            Leaf: class = 1 (gini = 0.0000)
            Leaf: class = 1 (gini = 0.0000)
          Node: X[0] < 52.500 (gini = 0.2667)
            Leaf: class = 0 (gini = 0.0000)
            Leaf: class = 0 (gini = 0.0000)
        Leaf: class = 0 (gini = 0.0000)
    Node: X[4] < 145.000 (gini = 0.0434)
      Leaf: class = 0 (gini = 0.0000)
      Node: X[0] < 67.500 (gini = 0.0303)
        Leaf: class = 1 (gini = 0.0000)
        Node: X[0] < 68.500 (gini = 0.0000)
          Leaf: class = 0 (gini = 0.0000)
          Leaf: class = 1 (gini = 0.0000)
```

Figura 3: Representación del árbol para el dataset heart.

```
Balanced Accuracy: 0.7361
Sensibilidad (clase 1): 0.6944
Especificidad (clase 0): 0.7778
```

Figura 4: Métricas de evaluación para el dataset heart.

```
Balanced Accuracy: 0.9171
Sensibilidad (clase 1): 0.9076
Especificidad (clase 0): 0.9267

Node: X[12] < 4.500 (gini = 0.3574)
  Node: X[2] < 3.500 (gini = 0.2825)
    Node: X[9] < 35.500 (gini = 0.1622)
      Node: X[0] < 57.500 (gini = 0.1491)
        Node: X[4] < 154.500 (gini = 0.0443)
          Node: X[0] < 46.500 (gini = 0.0000)
            Leaf: class = 0 (gini = 0.0000)
            Leaf: class = 1 (gini = 0.0000)
          Node: X[3] < 109.000 (gini = 0.0256)
            Leaf: class = 0 (gini = 0.0000)
            Leaf: class = 0 (gini = 0.0000)
        Node: X[1] < 0.500 (gini = 0.2652)
          Node: X[0] < 59.000 (gini = 0.0702)
            Leaf: class = 1 (gini = 0.0000)
            Leaf: class = 0 (gini = 0.0000)
          Node: X[4] < 259.000 (gini = 0.2286)
            Leaf: class = 1 (gini = 0.0000)
            Leaf: class = 1 (gini = 0.0000)
      Leaf: class = 1 (gini = 0.0000)
    Node: X[11] < 0.500 (gini = 0.2946)
      Node: X[3] < 145.000 (gini = 0.2047)
        Node: X[7] < 96.500 (gini = 0.0767)
          Leaf: class = 1 (gini = 0.0000)
          Node: X[4] < 316.500 (gini = 0.0556)
            Leaf: class = 0 (gini = 0.0000)
            Leaf: class = 0 (gini = 0.0000)
        Node: X[10] < 1.500 (gini = 0.2222)
          Node: X[6] < 1.000 (gini = 0.0000)
            Leaf: class = 0 (gini = 0.0000)
            Leaf: class = 1 (gini = 0.0000)
          Leaf: class = 1 (gini = 0.0000)
      Node: X[1] < 0.500 (gini = 0.1579)
        Node: X[9] < 10.500 (gini = 0.0000)
          Leaf: class = 0 (gini = 0.0000)
          Leaf: class = 1 (gini = 0.0000)
        Leaf: class = 1 (gini = 0.0000)
  Node: X[11] < 0.500 (gini = 0.3162)
    Node: X[8] < 0.500 (gini = 0.3836)
      Node: X[0] < 51.000 (gini = 0.3229)
        Node: X[2] < 3.500 (gini = 0.2762)
          Node: X[0] < 47.000 (gini = 0.0000)
            Leaf: class = 0 (gini = 0.0000)
            Leaf: class = 1 (gini = 0.0000)
          Node: X[3] < 114.000 (gini = 0.1429)
            Leaf: class = 0 (gini = 0.0000)
            Leaf: class = 1 (gini = 0.0000)
        Node: X[9] < 3.500 (gini = 0.1412)
          Node: X[0] < 55.500 (gini = 0.2667)
            Leaf: class = 0 (gini = 0.0000)
            Leaf: class = 1 (gini = 0.0000)
          Leaf: class = 0 (gini = 0.0000)
      Node: X[9] < 15.500 (gini = 0.2431)
        Node: X[4] < 255.000 (gini = 0.2381)
          Node: X[0] < 51.500 (gini = 0.1905)
            Leaf: class = 0 (gini = 0.0000)
            Leaf: class = 0 (gini = 0.0000)
          Leaf: class = 1 (gini = 0.0000)
        Leaf: class = 1 (gini = 0.0000)
    Node: X[3] < 109.000 (gini = 0.1356)
      Node: X[0] < 55.000 (gini = 0.0000)
        Leaf: class = 0 (gini = 0.0000)
        Leaf: class = 1 (gini = 0.0000)
      Node: X[4] < 145.000 (gini = 0.0920)
        Leaf: class = 0 (gini = 0.0000)
        Node: X[0] < 61.500 (gini = 0.0836)
          Leaf: class = 1 (gini = 0.0000)
          Node: X[6] < 1.000 (gini = 0.2000)
            Leaf: class = 0 (gini = 0.0000)
            Leaf: class = 1 (gini = 0.0000)
```

Figura 5: Representación del árbol y sus métricas de evaluación para el dataset completo.

LED Display Domain

```
Balanced Accuracy: 0.8920
Sensibilidad (clase 1): 0.8750
Especificidad (clase 0): 0.9091

Node: X[5] < 0.500 (gini = 0.0934)
  Node: X[6] < 0.500 (gini = 0.0794)
    Node: X[4] < 0.500 (gini = 0.2381)
      Leaf: class = 0 (gini = 0.0000)
      Node: X[0] < 0.500 (gini = 0.2500)
        Leaf: class = 1 (gini = 0.0000)
        Node: X[1] < 0.500 (gini = 0.3333)
          Node: X[3] < 0.500 (gini = 0.3333)
            Leaf: class = 1 (gini = 0.0000)
            Leaf: class = 1 (gini = 0.0000)
          Leaf: class = 1 (gini = 0.0000)
    Leaf: class = 1 (gini = 0.0000)
  Node: X[1] < 0.500 (gini = 0.0000)
    Leaf: class = 0 (gini = 0.0000)
    Leaf: class = 1 (gini = 0.0000)
```

Figura 6: Métricas de evaluación para el dataset heart.

```
Balanced Accuracy: 0.9706
Sensibilidad (clase 1): 0.9412
Especificidad (clase 0): 1.0000

Node: X[5] < 0.500 (gini = 0.1269)
  Node: X[6] < 0.500 (gini = 0.1006)
    Node: X[3] < 0.500 (gini = 0.2708)
      Node: X[0] < 0.500 (gini = 0.0000)
        Leaf: class = 0 (gini = 0.0000)
        Leaf: class = 1 (gini = 0.0000)
      Node: X[0] < 0.500 (gini = 0.2083)
        Leaf: class = 1 (gini = 0.0000)
        Node: X[1] < 0.500 (gini = 0.2667)
          Node: X[4] < 0.500 (gini = 0.3000)
            Leaf: class = 1 (gini = 0.0000)
            Leaf: class = 0 (gini = 0.0000)
          Leaf: class = 1 (gini = 0.0000)
    Leaf: class = 1 (gini = 0.0000)
  Node: X[3] < 0.500 (gini = 0.0381)
    Leaf: class = 0 (gini = 0.0000)
    Node: X[4] < 0.500 (gini = 0.0000)
      Leaf: class = 0 (gini = 0.0000)
      Leaf: class = 1 (gini = 0.0000)
```

Figura 7: Representación del árbol y sus métricas de evaluación para el dataset completo.

Anexo

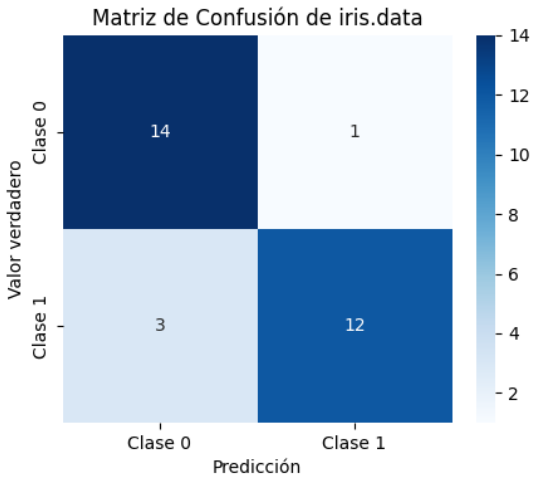

Figura 8: Matriz de confusión para iris.

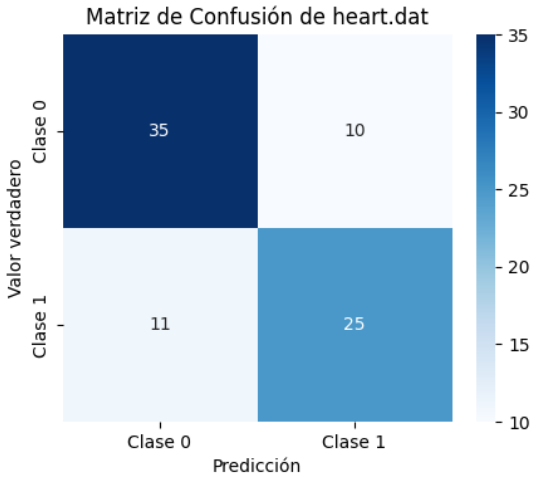

Figura 9: Matriz de confusión para heart disease.

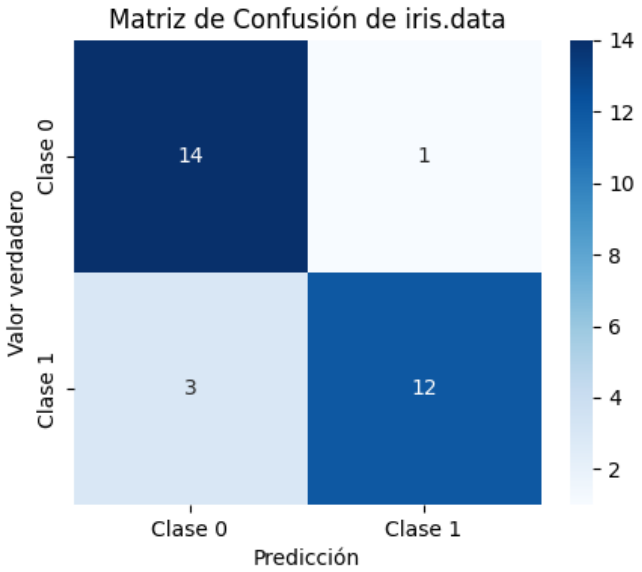

Figura 10: Matriz de confusión para LED display domain.

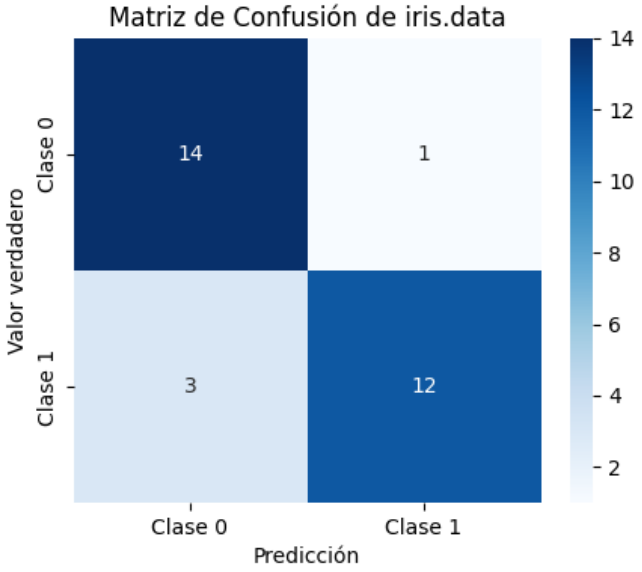

Figura 11: Matriz de confusión para iris dataset completo.

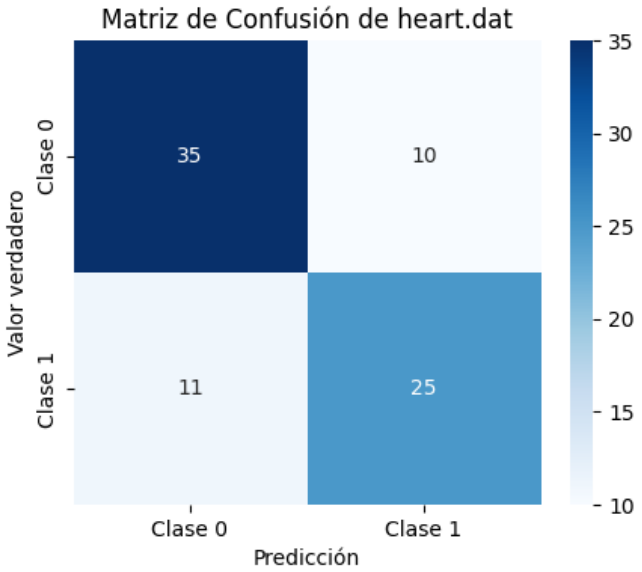

Figura 12: Matriz de confusión para heart disease dataset completo.

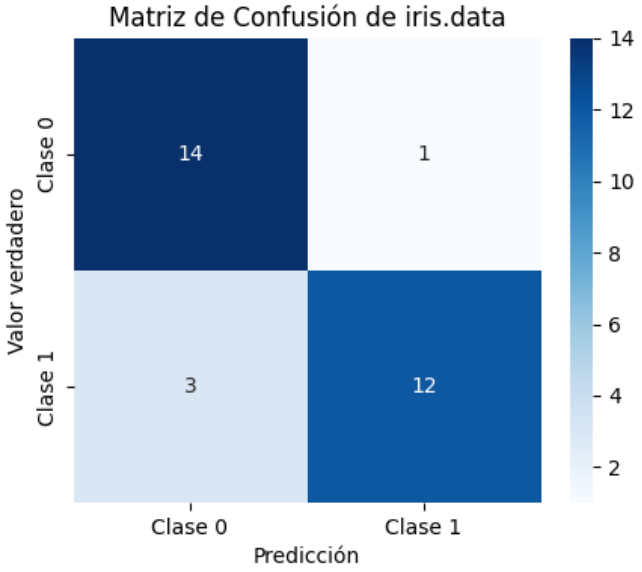

Figura 13: Matriz de confusión para LED display domain data set completo.